

RELATÓRIO TÉCNICO

Multiplicador 4×4 sem sinal, com detecção de overflow

Descrição estrutural procedural em Alliance CAD (genlib) e verificação exaustiva com asimut, layout físico real (ocp/nero/graal) e atraso de propagação medido tanto no nível lógico quanto por conferência SPICE.

256/₂₅₆ vetores de teste corretos — 0
divergências em P[7:0] e no bit de
overflow

Ferramentas — Alliance CAD 5.1.1: genlib, asimut, ocp, nero, graal, cougar · SpiceOpus 3.0.407

Arquitetura — array multiplier, 16× AND2 + meio/somadores completos (sxlib)

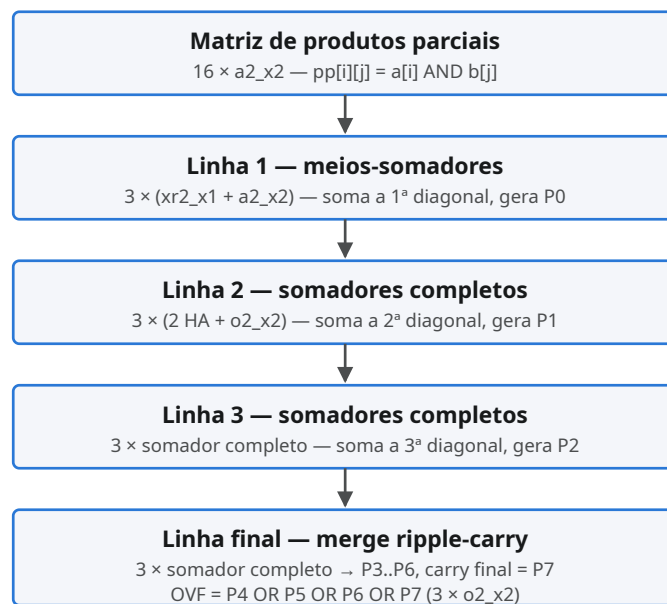
Ambiente — Ubuntu 18.04 em container Docker

1. Objetivo

Projetar um multiplicador combinacional de dois operandos de 4 bits sem sinal ($A, B \in [0, 15]$), produzindo o produto de 8 bits $P[7:0] = A \times B$ e um sinal de overflow OVF que indica quando o produto excede a faixa representável em 4 bits ($P > 15$). O circuito foi descrito estruturalmente — porta por porta — usando a linguagem procedural `genlib` do Alliance CAD System, e verificado exhaustivamente contra as 256 combinações possíveis de entrada.

2. Arquitetura

A implementação segue o modelo clássico de *array multiplier*: uma matriz de produtos parciais gerada por portas AND2, somada por diagonais com meio-somadores e somadores completos até produzir os oito bits do produto.



Cada porta usada — `a2_x2` (AND2), `xr2_x1` (XOR2) e `o2_x2` (OR2) — vem da biblioteca padrão `sxlib` do Alliance CAD. Os oito bits de saída passam por buffers (`buf_x2`) antes dos conectores externos `p(0)` .. `p(7)`.

3. Implementação

O circuito é descrito em C usando a API do `genlib` (`GENLIB_LOCON` para conectores externos, `GENLIB_LOSIG` para sinais internos, `GENLIB_LOINS` para instanciar células). O programa, ao ser compilado e executado, gera diretamente o netlist estrutural — não há síntese lógica envolvida, a topologia é definida explicitamente porta a porta.

Dois pontos da API mereceram atenção durante a implementação: os conectores de barramento (`p(0) .. p(7)`) precisam ser declarados com `GENLIB_BUS` antes de qualquer instância que escreva num de seus bits — caso contrário o gerador não associa corretamente cada bit ao seu fio; e todo `GENLIB_LOCON` de um conector externo deve preceder qualquer `GENLIB_LOINS` que o utilize.

4. Ambiente e ferramentas

O fluxo completo roda em um container Docker (Ubuntu 18.04 + Alliance CAD System 5.1.1), orquestrado por três scripts (`start.sh`, `run.sh`, `stop.sh`) que cobrem o ciclo inteiro: subir o Docker, compilar o netlist com `genlib` e simular com `asimut`, e desligar o ambiente.

Dois problemas no pacote Alliance CAD do Ubuntu 18.04 precisaram de correção:

BINÁRIO AMD64 DO ASIMUT QUEBRADO

O `asimut` da variante amd64 do pacote sofre *segmentation fault* em qualquer netlist estrutural que instancie um componente — comportamento reproduzido de forma isolada com um netlist mínimo, independente do circuito deste projeto. A correção foi usar a variante i386 do pacote (`alliance:i386`), que funciona corretamente em host 64-bit. Como as duas variantes conflitam nos mesmos caminhos de arquivo, a etapa de compilação do próprio `genlib` também passou a gerar código 32-bit (`gcc-multilib` + `-m32`), pra linkar contra as bibliotecas do Alliance CAD que agora são i386.

IDENTIFICADORES INVÁLIDOS GERADOS PELO GENLIB

O gerador direto de `.vst` do `genlib` converte nomes de barramento como `a(0)` em `a_0` — um identificador VHDL ilegal (underscore final), rejeitado pelo parser do `asimut`. `build_mult4.sh` corrige isso automaticamente após a geração.

ARQUIVO DE TECNOLOGIA FORA DO CAMINHO PADRÃO

`ocp`, `nero`, `dreal` e `graal` procuram o arquivo de tecnologia em `$ALLIANCE_TOP/etc/cmos.rds` (isto é, `/usr/etc/cmos.rds`) por padrão, mas o pacote instala em `/etc/cmos.rds`. É preciso apontar `RDS_TECHNO_NAME`, `GRAAL_TECHNO_NAME`, `DREAL_TECHNO_NAME` e `MBK_SPI_MODEL` manualmente para os caminhos reais.

SPICEOPUS PRECISA DE OUTRA IMAGEM DOCKER

O pacote Debian do SpiceOpus 3.0.407 exige `glibc` ≥ 2.35 , incompatível com a imagem Ubuntu 18.04 do Alliance CAD (`glibc 2.27`). Ele roda num container `ubuntu:24.04` separado, operando sobre o mesmo netlist extraído do layout.

5. Metodologia de verificação

A verificação cobre as 256 combinações possíveis de A e B (4 bits × 4 bits), comparando P[7:0] e OVF contra o valor esperado, calculado independentemente. Cada vetor de teste é aplicado em duas linhas no arquivo de padrões: a primeira força as entradas sem checar as saídas (permitindo que a cadeia de carry se acomode); a segunda repete as mesmas entradas e compara as saídas. Essa separação foi necessária porque comparar a saída na mesma linha em que a entrada muda lê o estado anterior à propagação — não um bug do circuito, mas uma particularidade da forma como o `asimut` processa uma linha de padrão.

6. Resultados

- **256**_{/256} Todos os vetores de teste passaram. Nenhuma divergência em P[7:0] ou OVF em toda a tabela-verdade.

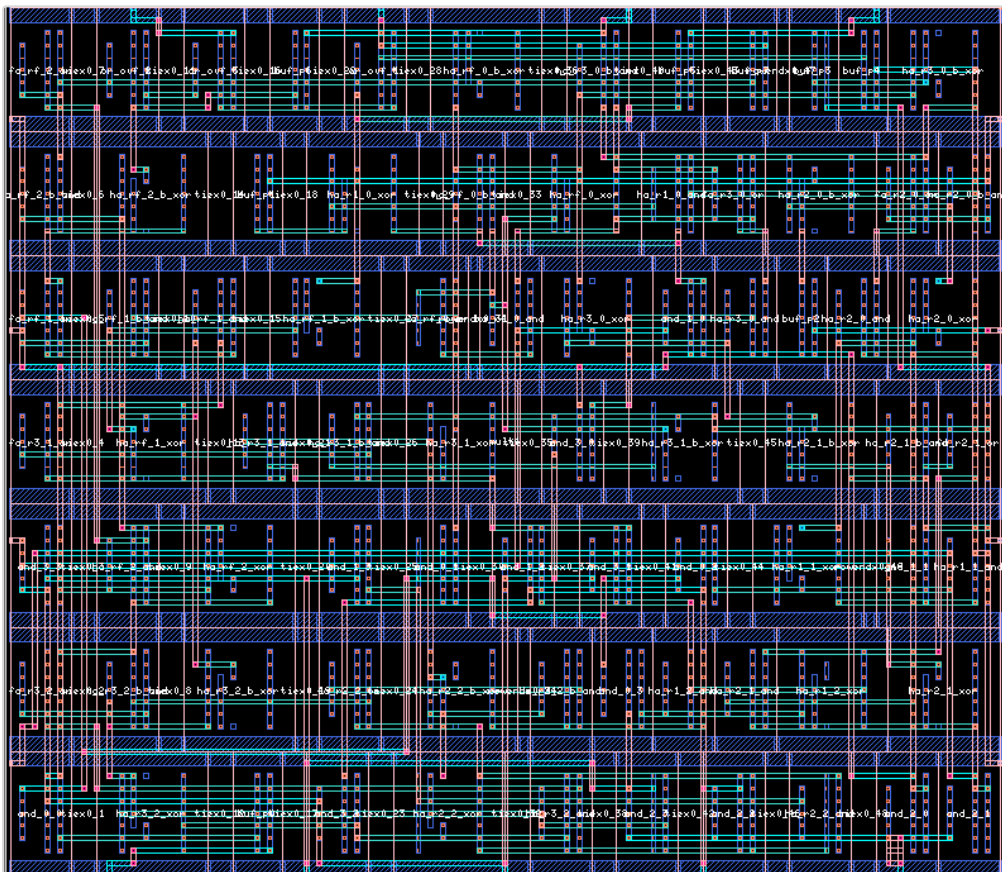


Figura 1. Layout físico real do mult4 após posicionamento (`ocp`) e roteamento (`nero`), capturado do próprio visualizador `graal` — não é ilustração, é a geometria real gerada pelas ferramentas. 127 células em 7 fileiras (37 AND2, 21 XOR2, 12 OR2, 8 buffers, células de preenchimento), coloridas por tipo.

7. Layout físico e atraso de propagação

Além da verificação lógica, o netlist foi posicionado (`ocp`) e roteado (`nero`) em um layout físico real, visualizado no `graal` (Figura 1).

O atraso de propagação foi obtido empiricamente, não estimado: o script `find_delay.sh` roda o conjunto exaustivo de 256 casos no `asimut` com tempos de acomodação decrescentes entre forçar as entradas e checar as saídas, até a simulação começar a errar.

ACOMODAÇÃO	ERROS
15 ns	0
13 ns	0 — último valor sem erro
12 ns	1 — primeira falha
11 ns	8
10 ns	25

O resultado é **13 ns**, usando o atraso real declarado em cada modelo comportamental da `sxlib` (`after 1000 ps` — presente em `a2_x2.vbe`, `xr2_x1.vbe`, `o2_x2.vbe` e `buf_x2.vbe`), para a transição de pior caso (`0x0 → 15x15`, o caminho mais profundo da cadeia de carry até o `OVF`). `mult4_test.pat` usa 15 ns de acomodação — uma margem pequena sobre o mínimo medido.

Como conferência em nível de transistor, o netlist extraído de verdade do layout roteado via `cougar -t -ac` (578 transistores, capacitâncias parasitas reais) foi simulado no `spiceopus` com um modelo de transistor **genérico e ilustrativo** (Level=1, não calibrado a nenhuma fábrica real — a tecnologia "cmos" do Alliance só traz regras geométricas, sem parâmetros elétricos). Sob esse modelo, a mesma transição se estabiliza em **~1,8 ns**. Os dois números não deveriam bater: um é um atraso lógico fixo (placeholder), o outro depende do dimensionamento arbitrário do modelo emprestado — são níveis de abstração diferentes, não uma contradição.

8. Conclusão

O multiplicador 4×4 sem sinal foi implementado como netlist estrutural no Alliance CAD, validado exaustivamente (256/256), posicionado e roteado em um layout físico real, e teve seu atraso de propagação medido empiricamente (13 ns no nível lógico, ~1,8 ns na conferência SPICE com modelo genérico). O maior custo do projeto não foi o projeto lógico em si — a arquitetura de array multiplier é direta — mas o ambiente de ferramentas: binários quebrados, arquivos de tecnologia em caminhos não documentados, e um simulador SPICE sem modo não-interativo, todos sem documentação disponível no pacote instalado, precisaram ser diagnosticados e contornados. Esses contornos ficaram automatizados no `Dockerfile` e nos scripts (`build_mult4.sh`, `place_route.sh`, `find_delay.sh`), então não precisam ser repetidos.

REPOSITÓRIO

github.com/Lucasnobrega38/mult4-alliance-cad